

AI Agents for DevOps - Structured Study Notes

Derived from the attached ChatGPT learning conversation. Completed work and planned future topics are intentionally distinguished.

1. Roadmap

LLM -> Prompting -> Tools -> Tool Calling -> Agents -> Agent Frameworks -> DevOps Agents -> Production

Core target: build a DevOps AI agent that can investigate AWS, GitHub Actions, Kubernetes, Terraform, monitoring and logs.

2. Core architecture

```
USER
|
v
AGENT
+--> MODEL (reasoning)
+--> TOOLS / FUNCTIONS
+--> MEMORY / CONTEXT
+--> PLANNING / STATE
|
v
AWS / GitHub / Kubernetes / Terraform / Monitoring
|
v
EVIDENCE -> MODEL -> DIAGNOSIS / ANSWER
```

Agent = definition/configuration and capabilities.

Model = reasoning/language engine.

Tool = callable real-world capability.

State = current evidence/context and workflow position.

Planner = next-action selection from current state.

3. Theory branch

AI is the broad field; **Generative AI** produces new content; **LLM** handles and generates language.

Text -> tokens -> token IDs -> model processing -> next-token prediction -> generated response

Token = text piece. **Token ID** = numeric identifier. **Context window** = token budget for current context.

Messages: system = behavior/instructions; user = request; assistant = response; tool call/result = external-function exchange.

Temperature controls variation. **Hallucination** means plausible but incorrect information. **Grounding** means anchoring output to real context, tools and validation.

Memory link: short-term memory <-> conversation history <-> context window. Long-term memory and RAG were planned later in the source course.

4. Tool calling

```
1. User request
2. Call model
3. Inspect response.output
4. Detect item.type == "function_call"
5. Read name / arguments / call_id
6. Execute Python tool
7. Return function_call_output with same call_id
8. Call model again
9. Repeat until final answer

for item in response.output:
    if item.type == "function_call":
        print("LLM wants to call a tool")
        print("Tool:", item.name)
        print("Arguments:", item.arguments)
        print("Call ID:", item.call_id)
```

call_id links a tool result to the exact tool request. **function_call_output** sends the result back so the model can continue deciding.

5. Agent loop

```
User
-> LLM
-> Need more information?
    -> YES -> Tool -> Result -> LLM
    -> NO -> Final answer
```

The model may be called multiple times because each new evidence result can change the next decision.

```
Example:
ECS service unhealthy
-> get service status
-> get tasks
-> get task details
-> get CloudWatch logs
-> diagnose
```

6. State and planning

The state built in the conversation evolved to include cluster, service, service_status, tasks, stopped_tasks, task_details, service events, CloudTrail, CloudWatch logs, failure classification, evidence analysis, investigation status, missing evidence and next action.

```
Current state
|
v
determine_next_action(state)
|
+--> collect evidence
+--> recheck
+--> analyze evidence
+--> determine missing evidence
+--> diagnose
+--> complete
```

Infinite-loop lesson: if the planner returns the same action without changing relevant state, the loop can repeat forever. Every action needs a meaningful state transition or terminal condition.

7. ECS troubleshooting evidence chain

1. ECS service status
2. ECS tasks
3. stopped-task details
4. service events
5. CloudTrail
6. CloudWatch log configuration
7. exact log stream
8. log events
9. deterministic evidence analysis
10. failure classification
11. incident decision
12. LLM diagnosis

ECS lifecycle refinement: desired_count == running_count was used as a capacity/recovery check for the workflow, but desired_count == 0 can be intentional and a user-initiated stop is not automatically an application failure.

8. CloudWatch correlation

```
task ARN
-> task definition / log configuration
-> log group + stream prefix
-> exact log stream
-> log events
-> evidence analysis
```

The conversation tested CloudWatch pieces independently before integration. This exposed naming assumptions and lifecycle situations where a task had not yet produced a usable log stream.

9. Failure classification and evidence correlation

```
failure_stage / failure_type

task_startup / ECR_CONNECTIVITY
task_startup / ECR_AUTHORIZATION
task_termination / USER_INITIATED_STOP

Classification
-> evidence validation
-> conflict detection
-> confidence
-> diagnosis
```

Example conflict: ECS reports USER_INITIATED_STOP while a container has a non-zero exit code. The workflow can retain the ECS classification and record evidence_conflict = true.

10. Incident decision engine

```
Collect -> Analyze -> Complete?
      |
      | no | yes
      |   |
      |   | -> Diagnose
      |   |
      |   v
      | determine_missing_evidence
      |
      | v
      | choose next tool
      |
      +----> Collect more evidence
```

The missing-evidence branch replaces a vague 'investigate_more' step with targeted selection of what evidence is still needed.

11. OpenAI Agents SDK

Component	Memorize	Typical syntax
Agent	Definition	Agent(name=..., model=..., tools=[...])
Model	Reasoning	model="gpt-5.4-mini"
Runner	Execution	Runner.run_sync(agent, request)

```
from agents import Agent, Runner, function_tool

@function_tool
def get_environment():
    return {"environment": "dev", "region": "eu-west-1"}

agent = Agent(
    name="ECS DevOps Agent",
    model="gpt-5.4-mini",
    instructions="Use tools for real AWS state; do not invent it.",
    tools=[get_ecs_service_status],
)

result = Runner.run_sync(
    agent,
    "Check the ECS service status."
)
```

@function_tool exposes a Python function as a callable tool and supplies the interface/schema the runtime can use.

Multiple tools: the model selects from the available tool definitions. **Dependent tools:** one tool's output supplies the input for the next.

```
get_ecs_tasks()
->
select first task
->
get_ecs_task_details()
->
final answer
```

12. Framework and planned roadmap

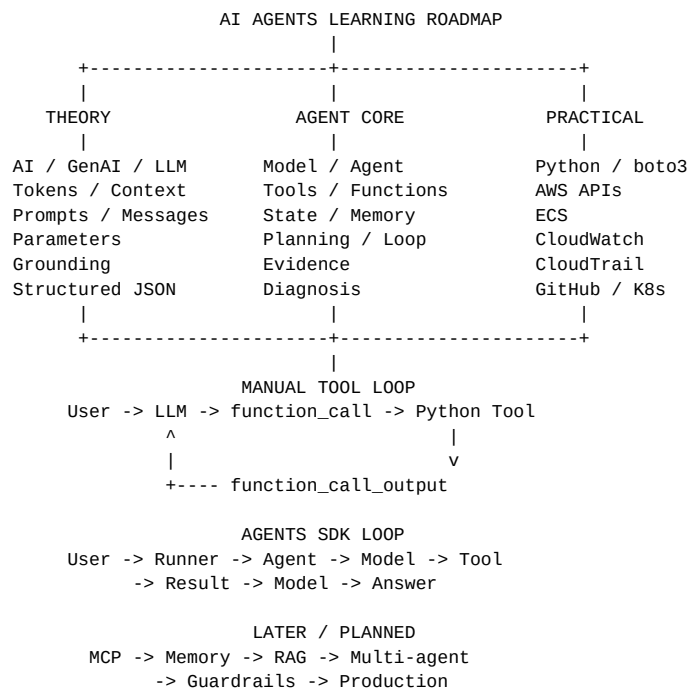
Source roadmap: OpenAI Agents SDK -> Anthropic -> LangChain -> LangGraph -> MCP -> possibly other frameworks where useful.

```
AI Agent
|
v
MCP
+--> AWS
+--> GitHub
+--> Kubernetes
```

+-> Terraform
+-> Databases
+-> Monitoring

Planned later: Memory, RAG, Multi-agent systems, Guardrails/Security, Production architecture.

13. Final single-glance map



14. Memorize these

Agent = Definition

Model = Reasoning

Runner = Execution

Manual: Model -> function call -> Python tool -> function call output -> Model

DevOps: Incident -> Evidence -> State -> Planner -> Tool -> Analyze -> Diagnose

Correctness: Never let the LLM invent infrastructure state when a tool can retrieve the real state.

15. Completed progression visible in the chat

Single-turn API -> context experiments -> multi-turn history -> JSON -> function calling -> call_id -> function_call_output -> full tool loop -> boto3/AWS -> ECS tools -> state/planning -> CloudTrail -> CloudWatch -> evidence correlation -> failure classification -> incident decision engine -> investigation loop -> missing-evidence branch -> Agents SDK -> Agent/Model/Runner -> @function_tool -> multiple tools -> dependent tools -> tracing.

Source note: the guide intentionally does not treat future roadmap items as completed implementations.